Name: Alegre, Jhon Isaac S.	**Date Performed: 01/08/2026**
Course/Section: CPE212-CPE31S1	**Date Submitted: 01/08/2026**
Instructor: Engr. Robin Valenzuela	**Semester and SY: 2025-2026**

Activity 4: Running Elevated Ad hoc Commands

1. **Objectives:**

1.1 Use commands that makes changes to remote machines

1.2 Use playbook in automating ansible commands

2. **Discussion:**

Provide screenshots for each task.

Elevated Ad hoc commands

So far, we have not performed ansible commands that makes changes to the remote servers. We manage to gather facts and connect to the remote machines, but we still did not make changes on those machines. In this activity, we will learn to use commands that would install, update, and upgrade packages in the remote machines. We will also create a playbook that will be used for automations.

Playbooks record and execute **Ansible**'s configuration, deployment, and orchestration functions. They can describe a policy you want your remote systems to enforce, or a set of steps in a general IT process. If Ansible modules are the tools in your workshop, playbooks are your instruction manuals, and your inventory of hosts are your raw material. At a basic level, playbooks can be used to manage configurations of and deployments to remote machines. At a more advanced level, they can sequence multi-tier rollouts involving rolling updates, and can delegate actions to other hosts, interacting with monitoring servers and load balancers along the way. You can check this documentation if you want to learn more about playbooks. Working with playbooks — Ansible Documentation

Task 1: Run elevated ad hoc commands

1. Locally, we use the command *sudo apt update* when we want to download package information from all configured resources. The sources often defined in /etc/apt/sources.list file and other files located in /etc/apt/sources.list.d/ directory. So, when you run update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run

an apt update command in a remote machine. Issue the following command:

ansible all -m apt -a update_cache=true

What is the result of the command? Is it successful? **Yes**

```
saac@Workstation:~/ansible$ ansible all -m apt -a update_cache=true -b -K
ECOME password:
92.168.56.104 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1767864305,
    "cache_updated": true,
    "changed": true

92.168.56.108 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1767864305,
    "cache_updated": true,
    "changed": true

saac@Workstation:~/ansible$
```

Try editing the command and add something that would elevate the privilege. Issue the command *ansible all -m apt -a update_cache=true --become --ask-become-pass.* Enter the sudo password when prompted. You will notice now that the output of this command is a success. The *update_cache=true* is the same thing as running *sudo apt update*. The --become command elevate the privileges and the *--ask-become-pass* asks for the password. For now, even if we only have changed the packaged index, we were able to change something on the remote server.

You may notice after the second command was executed, the status is CHANGED compared to the first command, which is FAILED.

```
saac@Workstation:~/ansible$ ansible all -m apt -a update_cache=true -b -K
ECOME password:
92.168.56.104 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1767864305,
    "cache_updated": true,
    "changed": true

92.168.56.108 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1767864305,
    "cache_updated": true,
    "changed": true

saac@Workstation:~/ansible$
```

2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: *ansible all -m apt -a name=vim-nox --become --ask-become-pass.* The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.
   2.1 Verify that you have installed the package in the remote servers. Issue the command *which vim* and the command *apt search vim-nox* respectively. Was the command successful?
   2.2 Check the logs in the servers using the following commands: *cd /var/log*. After this, issue the command *ls,* go to the folder *apt* and open history.log. Describe what you see in the history.log.

3. This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

   3.1 Issue the command: *ansible all -m apt -a name=snapd --become --ask-become-pass*

```
isaac@Workstation:~/ansible$ ansible all -m apt -a name=snapd --becom
ome-pass
BECOME password:
192.168.56.108 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1767864712,
    "cache_updated": false,
    "changed": false
}
192.168.56.104 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1767864714,
    "cache_updated": false,
    "changed": false
}
isaac@Workstation:~/ansible$
```

Can you describe the result of this command? Is it a success? Did it change anything in the remote servers? **Yes, it's a succes, it just discover the python.**

3.2 Now, try to issue this command: *ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass*
Describe the output of this command. Notice how we added the command *state=latest* and placed them in double quotations.

4. At this point, make sure to commit all changes to GitHub.

Task 2: Writing our First Playbook

1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (*CPE232_yourname*). Issue the command *nano install_apache.yml*. This will create a playbook file called *install_apache.yml*. The .yml is the basic standard extension for playbook files.

When the editor appears, type the following:

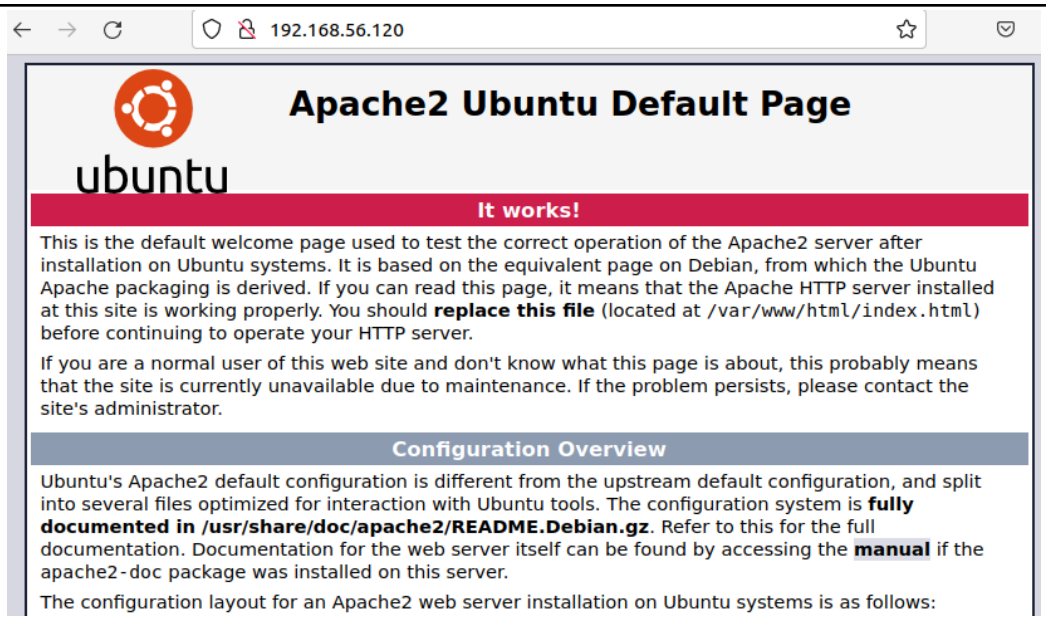
```
  GNU nano 4.8                    install_apache.yml
---
- hosts: all
  become: true
  tasks:

  - name: install apache2 package
    apt:
      name: apache2
```

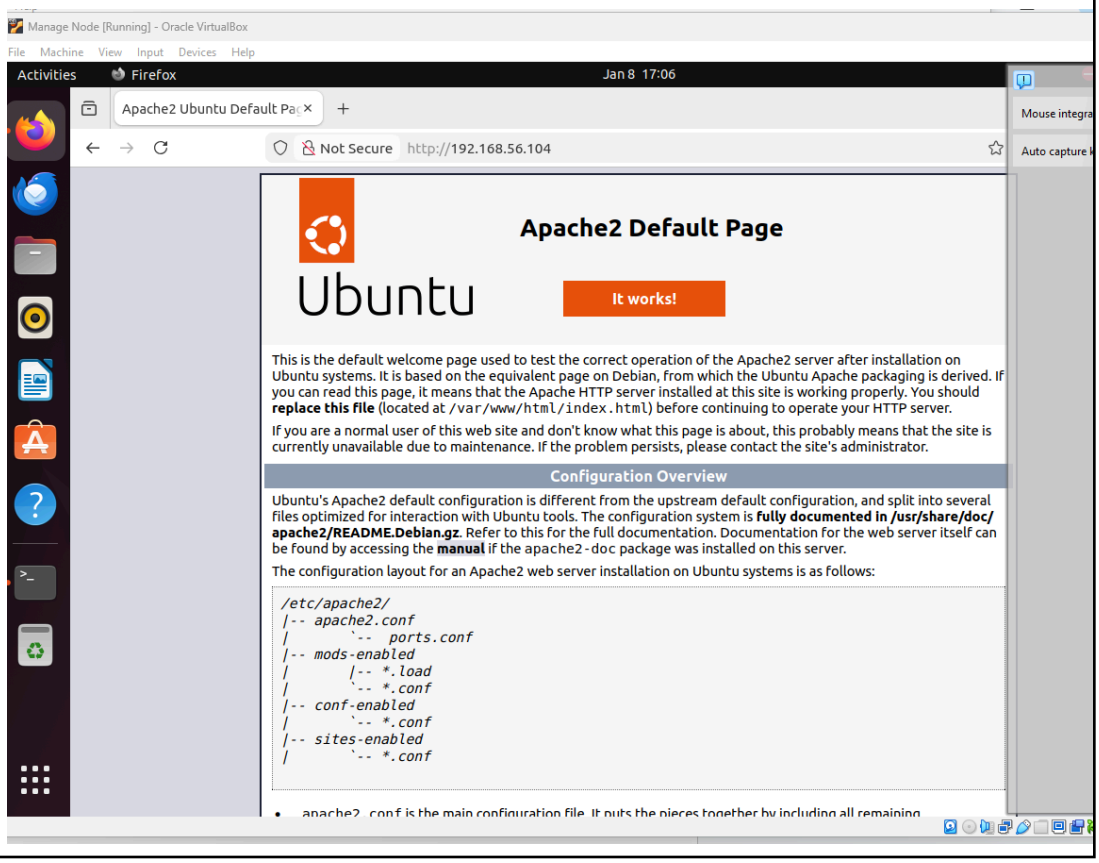
Make sure to save the file. Take note also of the alignments of the texts.

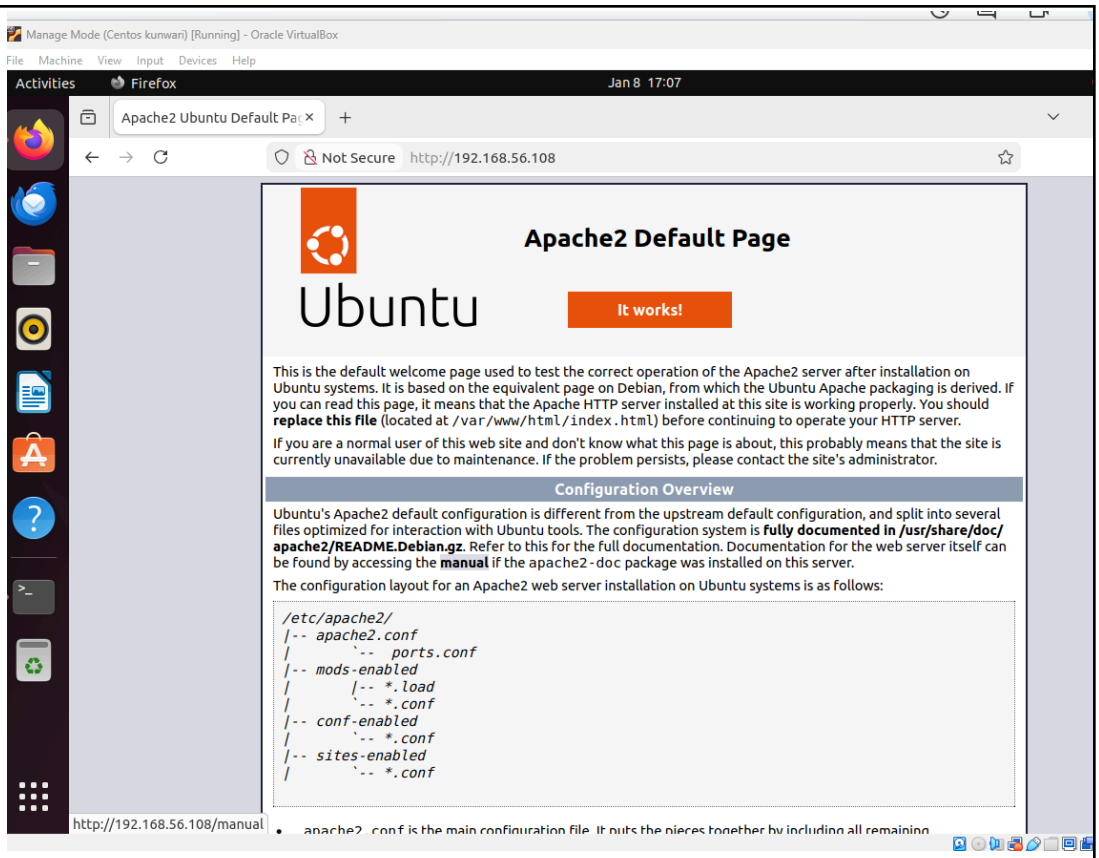
2. Run the yml file using the command: *ansible-playbook --ask-become-pass install_apache.yml.* Describe the result of this command.

```
isaac@Workstation:~/ansible$ ansible-playbook ./install_apache.yml -K
BECOME password:

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [192.168.56.108]
ok: [192.168.56.104]

TASK [install apache2 package] *************************************************
ok: [192.168.56.108]
ok: [192.168.56.104]

PLAY RECAP *********************************************************************
192.168.56.104             : ok=2    changed=0    unreachable=0    failed=0    skipped=0    res
  ignored=0
192.168.56.108             : ok=2    changed=0    unreachable=0    failed=0    skipped=0    res
  ignored=0

isaac@Workstation:~/ansible$
```

3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.


192.168.56.120
ubuntu
Apache2 Ubuntu Default Page
It works!
This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should replace this file (located at /var/www/html/index.html) before continuing to operate your HTTP server.
If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.
Configuration Overview
Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is fully documented in /usr/share/doc/apache2/README.Debian.gz. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the manual if the apache2-doc package was installed on this server.
The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:


Manage Node [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities
Firefox
Jan 8 17:06
Apache2 Ubuntu Default Pa
Not Secure
http://192.168.56.104
Ubuntu
Apache2 Default Page
It works!
This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should replace this file (located at /var/www/html/index.html) before continuing to operate your HTTP server.
If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.
Configuration Overview
Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is fully documented in /usr/share/doc/apache2/README.Debian.gz. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the manual if the apache2-doc package was installed on this server.
The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:
/etc/apache2/
|-- apache2.conf
| `-- ports.conf
|-- mods-enabled
| |-- *.load
| `-- *.conf
|-- conf-enabled
| `-- *.conf
|-- sites-enabled
| `-- *.conf
Mouse integra
Auto capture



4. Try to edit the *install_apache.yml* and change the name of the package to any name that will not be recognized. What is the output?
5. This time, we are going to put additional task to our playbook. Edit the *install_apache.yml*. As you can see, we are now adding an additional command, which is the *update_cache*. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.

```
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes

  - name: install apache2 package
    apt:
      name: apache2
```

Save the changes to this file and exit.

6. Run the playbook and describe the output. Did the new command change anything on the remote servers? **Yes, as you can see there's 1 changes for both server.**

```
isaac@Workstation:~/ansible$ ansible-playbook ./playbook.yml -K

TASK [Gathering Facts] *********************************************************
ok: [192.168.56.108]
ok: [192.168.56.104]

TASK [update repository index] *************************************************
changed: [192.168.56.104]
changed: [192.168.56.108]

TASK [install apache2 package] *************************************************
ok: [192.168.56.108]
ok: [192.168.56.104]

PLAY RECAP *********************************************************************
192.168.56.104             : ok=3    changed=1    unreachable=0    failed=0    skipped=0
  ignored=0
192.168.56.108             : ok=3    changed=1    unreachable=0    failed=0    skipped=0
  ignored=0

isaac@Workstation:~/ansible$
```

7. Edit again the *install_apache.yml*. This time, we are going to add a PHP support for the apache package we installed earlier.

```
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes

  - name: install apache2 package
    apt:
      name: apache2

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
```

Save the changes to this file and exit.

8. Run the playbook and describe the output. Did the new command change anything on the remote servers? **Yes**

```
isaac@Workstation:~/ansible$ ansible-playbook ./playbook.yml -K
BECOME password:

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [192.168.56.104]
ok: [192.168.56.108]

TASK [add PHP support for apache] **********************************************
ok: [192.168.56.104]
ok: [192.168.56.108]

TASK [update repository index] *************************************************
changed: [192.168.56.108]
changed: [192.168.56.104]

TASK [install apache2 package] *************************************************
ok: [192.168.56.104]
ok: [192.168.56.108]

PLAY RECAP *********************************************************************
192.168.56.104             : ok=4    changed=1    unreachable=0    failed=0    skipped=0
  ignored=0
192.168.56.108             : ok=4    changed=1    unreachable=0    failed=0    skipped=0
  ignored=0

isaac@Workstation:~/ansible$
```

9. Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository.

   [GitHub - EngrAlegre/CPE212_Alegre]

Reflections:

Answer the following:

1. What is the importance of using a playbook? **For efficiency and can save time for the one who manages/configuring the server. Playbooks in Ansible are essential for automating complex configuration, deployment, and orchestration tasks across multiple remote servers. They provide a declarative way to define the desired state of systems, ensuring consistency and repeatability that ad hoc commands lack.**
2. Summarize what we have done on this activity.

   In Activity 4, students ran elevated Ansible ad hoc commands like ansible all -m apt -a "update_cache=true" --become --ask-become-pass to update package caches, install vim-nox and snapd on remote Ubuntu servers, and verify changes through logs in /var/log/apt/history.log. They then created and refined the installapache.yml playbook to automate Apache2 installation, cache updates, and PHP support, observing "CHANGED" statuses and syncing everything to GitHub, demonstrating Ansible's power

for consistent, scalable server automation over one-off commands.